

Exercise on Convolutional Neural Networks

As we saw during the last sections, CNNs are a class of neural networks able to elaborate structured data such as images. To understand how a CNN works, we aim to build a simple classifier for the recognition of simple images.

Goal: Build a CNN that recognizes if an image contains a rectangle or a circle.

Let's start by importing some generic useful libraries and packages.

```
In [1]: import numpy as np
        from math import *
        from matplotlib import pyplot as plt
```

We now generate by ourselves some images with a circle or a rectangle, of random color, in a random position.

As said, images are matrices/tensors with usually 3 indices that regulate their size: height, length and color channels. In the so-called RGB images, the color channels are Red, Green and Blue and their values simply define the colors of the object. In our case, they define the color of the circles and of the rectangles. 2D Medical images are usually gray-scale images, that means that the color channel is one and only one.

Three different modes of generating the images are implemented:

- A single shape per figure
- Multiple shapes mixed in each figure (2nd part)
- A single figure keeping track of the "bounding box" (3rd part)

First, we write some functions that:

1. Define the background
2. Define the color
3. Draw a circle
4. Draw a rectangle
5. Generates N samples with the circle/rectangle using the two previous functions with correct targets

```
In [2]: import cv2

def background():
    # Here you may add noise
    return np.zeros((64,64,3), np.uint8)

def randomColor():
    return (int(np.random.rand()*128+128),int(np.random.rand()*128+128),int(np.random.rand()*128+128))

def drawCircle(c,x,y,r):
    img = background()
    cv2.circle(img,(x,y),r,c, -1)
    return img,x-r,y-r,x+r,y+r

def genCircle():
    return drawCircle(randomColor(),int(np.random.rand()*50)+10,int(np.random.rand()*50)+10,
                      int(np.random.rand()*6)+3)

def drawRectangle(c,x,y,w,h):
    img = background()
    cv2.rectangle(img,(x,y),((x+w),(y+h)), c, -1)
    return img,x,y,x+w,y+h

def genRectangle():
    return drawRectangle(randomColor(),int(np.random.rand()*40)+10,int(np.random.rand()*40)+10,
                        int(np.random.rand()*12)+5,int(np.random.rand()*12)+5)

nsamples=10000

# Produce figures with either a rectangle or a circle
targets=np.random.rand(nsamples) > 0.5
images=np.array([genCircle()[0] if targets[x] else genRectangle()[0] for x in range(nsamples)])
```

Circles have label 1 (True), Rectangles have label 0 (False). Let's show a few of the generated images.

```
In [3]: # Print target shape, images shape and the first 3 Labels/targets
print("Target shape:", np.shape(targets))
print("Images shape:", np.shape(images))
print("First 3 labels:", targets[:3])

# Plot the first three images and look if the targets correspond to the images
print(targets[0])
plt.imshow(images[0])
plt.show()

print(targets[1])
plt.imshow(images[1])
plt.show()
```

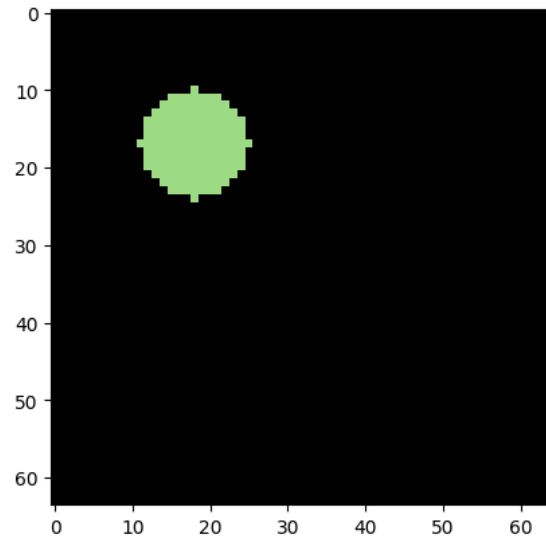
```
print(targets[2])  
plt.imshow(images[2])  
plt.show()
```

Target shape: (10000,)

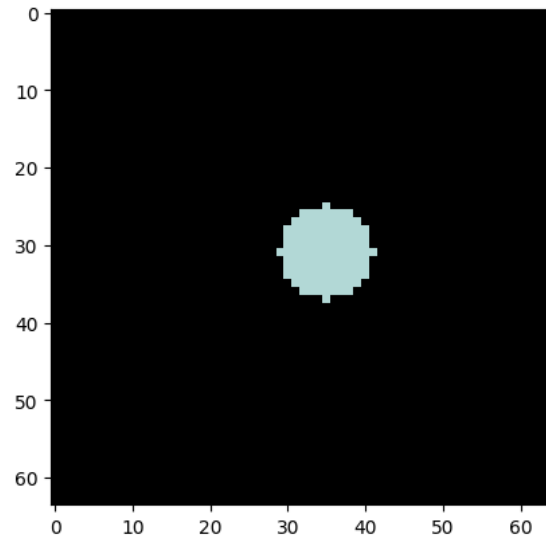
Images shape: (10000, 64, 64, 3)

First 3 labels: [True True True]

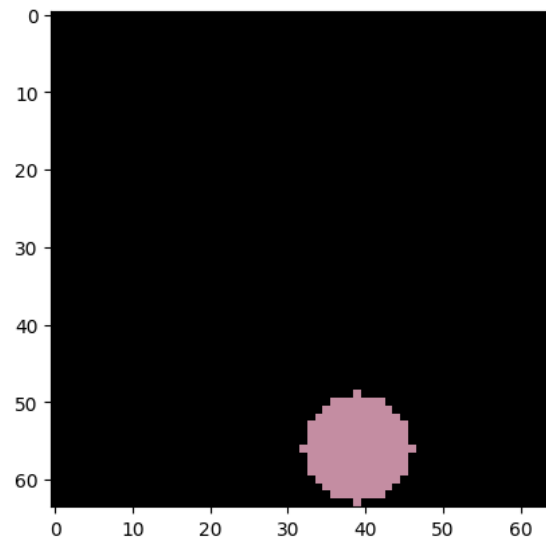
True



True



True



Additional Exercise

You can try this exercise once you have finished the main notebook.

1. Try adding some random noise in the image background
2. Try adding more classes such e.g. Lines or Ellipses

Now to build our first CNN we need to proceed through the following steps:

1. Import the correct layers from keras;
2. Import and define the Model we want to develop;
3. Compile the model;
4. Print a summary of the model and show its draw.

Please ALWAYS refer to the documentation if you have some doubt: <https://keras.io/api/>

As an exercise you can also try to implement these operations:

5. Try adding/removing convolutional layers, change the kernel size, try to add dropout.
6. Try changing the model to categorical labels, change loss function from binary_crossentropy to categorical_crossentropy, and use softmax activation instead of sigmoid.

```
In [4]: from keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense
from keras.models import Model
import tensorflow as tf

def SimpleModel():
    # Input Layer
    inputs = Input(shape=(64, 64, 3))

    # Hidden Layers
    # Convolutional
    x = Conv2D(filters=5, kernel_size=(5, 5), activation='relu')(inputs)

    # Pooling
    x = MaxPooling2D(pool_size=(3, 3))(x)

    # Convolutional
    x = Conv2D(filters=3, kernel_size=(3, 3), activation='relu')(x)

    # Pooling
    x = MaxPooling2D(pool_size=(3, 3))(x)

    # Flattening for the first dense Layer
    x = Flatten()(x)

    # Dense (Output shape: 20)
    hidden = Dense(units=20, activation='relu')(x)

    # Output Layer (Output shape: 1)
    outputs = Dense(units=1, activation='sigmoid')(hidden)

    simple_model = Model(inputs=inputs, outputs=outputs)
    simple_model.compile(loss='binary_crossentropy',
                        optimizer='adam',
                        metrics=['accuracy'])

    return simple_model

SimModel = SimpleModel()
SimModel.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 64, 64, 3)	0
conv2d (Conv2D)	(None, 60, 60, 5)	380
max_pooling2d (MaxPooling2D)	(None, 20, 20, 5)	0
conv2d_1 (Conv2D)	(None, 18, 18, 3)	138
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 3)	0
flatten (Flatten)	(None, 108)	0
dense (Dense)	(None, 20)	2,180
dense_1 (Dense)	(None, 1)	21

Total params: 2,719 (10.62 KB)

Trainable params: 2,719 (10.62 KB)

Non-trainable params: 0 (0.00 B)

Let's train our CNN splitting the sample in two so that 50% of it is used for validation and the other half for training.

```
In [5]: train_output=SimModel.fit(images, targets, validation_split=0.5, epochs=50, verbose=0)
```

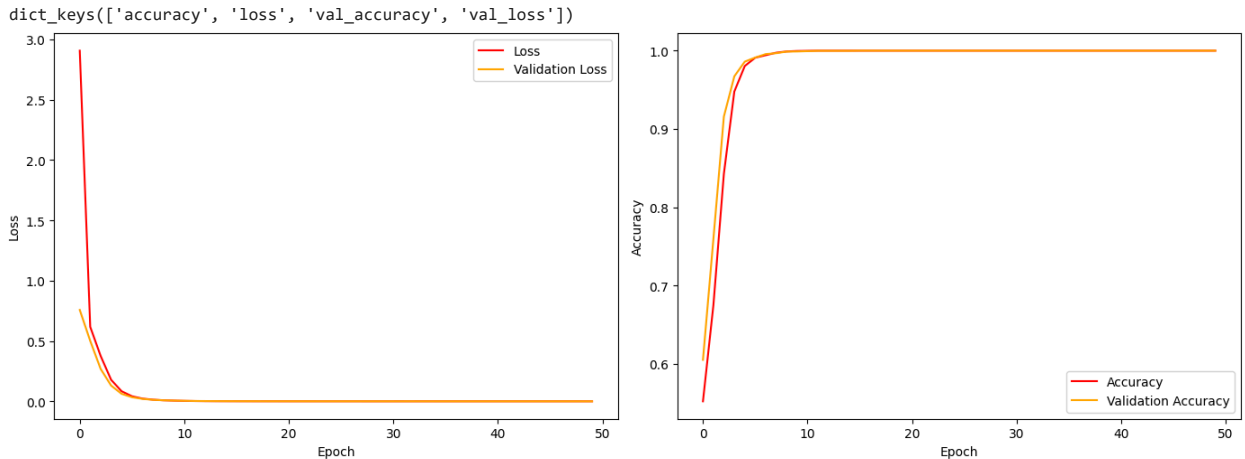
`train_output` contains information about the training. We can now show the loss vs epoch for both validation and training samples.

```
In [6]: print(train_output.history.keys())

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(train_output.history["loss"], color="red", label="Loss")
ax1.plot(train_output.history["val_loss"], color="orange", label="Validation Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
#ax1.set_yscale('log')
ax1.legend()

ax2.plot(train_output.history["accuracy"], color="red", label="Accuracy")
ax2.plot(train_output.history["val_accuracy"], color="orange", label="Validation Accuracy")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Accuracy")
ax2.legend()

plt.tight_layout()
plt.show()
```



You can see some signs of overfitting, but this problem is so simple for a CNN that it won't be an issue.

Note: Try to run the fitting with 100 epochs or increase the number of parameters. You will find that our model will be evidently overfitting, since the validation error will be much higher than the training one.

Now we generate one more circle and one more rectangle and we use the CNN to predict their class.

```
In [7]: test_rectangle = SimModel.predict(np.expand_dims(genRectangle()[0],axis=0) )
test_circle = SimModel.predict(np.expand_dims(genCircle()[0],axis=0) )

print('The predicted label for test_rectangle is', np.round((test_rectangle)))
print('The predicted label for test_circle is', np.round((test_circle)))

1/1 ----- 0s 378ms/step
1/1 ----- 0s 38ms/step
The predicted label for test_rectangle is [[0.]]
The predicted label for test_circle is [[1.]]
```

Better Data Management with `train_test_split`

Until now, our data management had a few methodological flaws:

- *Statistically weak testing*: Evaluating the model on just one random circle and one rectangle doesn't give a reliable measure of its real accuracy.
- *No isolated Test Set*: We used Keras' `validation_split` during training, but best practices require a completely unseen, dedicated *test set* to evaluate the final performance impartially.
- *No shuffling guarantees*: `validation_split` simply takes the last fraction of the dataset without mixing it. If our data had been sorted by class, the model would have failed to learn properly.

To fix this, we use `scikit-learn`'s `train_test_split` (https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html). This function automatically *shuffles* the data and randomly divides it into distinct subsets (training and test sets), ensuring a robust, reliable, and statistically valid training process.

```
In [8]: from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

Note: Here I'm still going to use `validation_split=0.2` to divide a validation set and a training while maintaining `x_test` intact for the final evaluation.

```
In [9]: # Fixed the random state for reproducibility
x_train, x_test, y_train, y_test = train_test_split(images, targets, test_size=0.2, random_state=42)
```

```
SimModelCorr = SimpleModel()
```

```
In [10]: train_output_corr = SimModelCorr.fit(x_train, y_train, epochs=50, validation_split=0.2, verbose=0)
```

```
In [11]: prediction_on_test = SimModelCorr.predict(x_test)
classification_performance = classification_report(y_test, np.round(prediction_on_test))
print(classification_performance)
```

```
63/63 ----- 1s 9ms/step
              precision    recall  f1-score   support

      False      1.00      1.00      1.00     1005
      True       1.00      1.00      1.00      995

 accuracy
macro avg      1.00      1.00      1.00     2000
weighted avg    1.00      1.00      1.00     2000
```

CNN for regression: how many rectangles and circles are on the image?

We have seen the simplest case of a binary classification before. Now, we want to generate images where there are both rectangles and circles and to train a neural network that is able to give as answer the number of circles and rectangles in the image.

First of all, we need to generate the images, using the functions that we previously defined.

```
In [12]: def genN(f,i):
img = np.zeros((64,64,3), np.uint8)
for x in range(i):
    img+=f()[0] # discard bb info, take only image
return img

targets=np.array([(int(np.random.rand()*4),int(np.random.rand()*4)) for x in range(nsamples) ])
images=np.array([genN(genRectangle,targets[x,0])+genN(genCircle,targets[x,1]) for x in range(nsamples)])
```

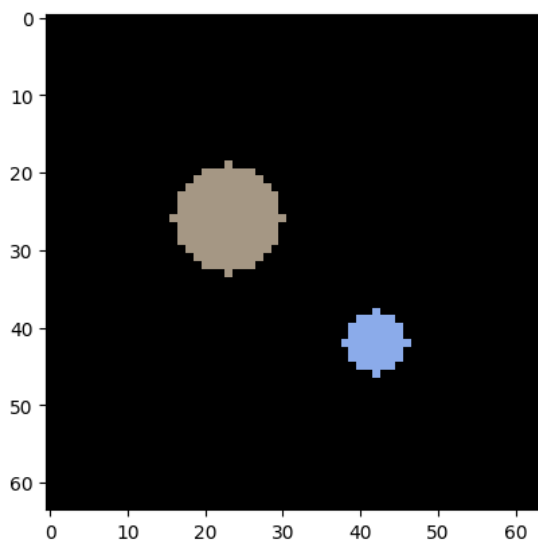
```
In [13]: # Print target shape, images shape and the first 3 labels/targets
print("Target shape:", np.shape(targets))
print("Images shape:", np.shape(images))
print("First 3 labels:\n", targets[:3])

# Plot the first three images and look if the targets correspond to the images
print("Target 0:", targets[0])
plt.imshow(images[0])
plt.show()

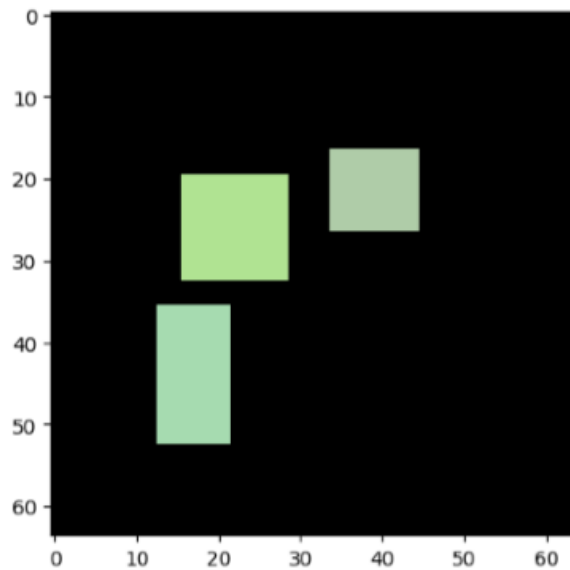
print("Target 1:", targets[1])
plt.imshow(images[1])
plt.show()

print("Target 2:", targets[2])
plt.imshow(images[2])
plt.show()
```

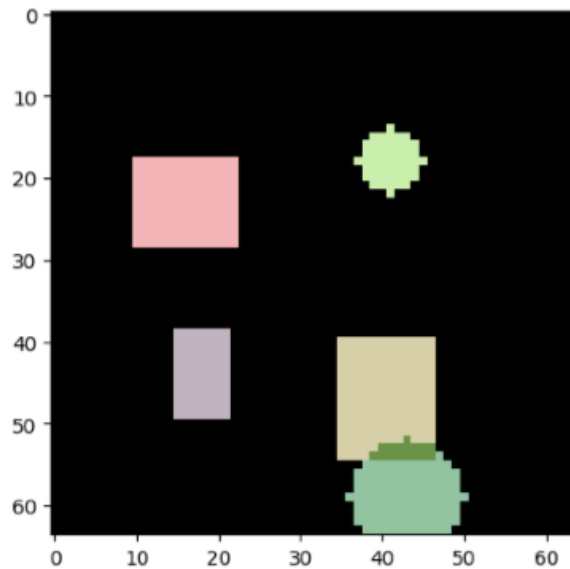
```
Target shape: (10000, 2)
Images shape: (10000, 64, 64, 3)
First 3 labels:
[[0 2]
 [3 0]
 [3 2]]
Target 0: [0 2]
```



```
Target 1: [3 0]
```



Target 2: [3 2]



Let's build the CNN.

```
] def CounterModel():
    # Input Layer
    inputs = Input(shape=(64, 64, 3))

    # Hidden Layers
    x = Conv2D(filters=5, kernel_size=(5, 5), activation='relu')(inputs)
    x = MaxPooling2D(pool_size=(3, 3))(x)
    x = Conv2D(filters=3, kernel_size=(3, 3), activation='relu')(x)
    x = MaxPooling2D(pool_size=(3, 3))(x)
    x = Flatten()(x)

    # Dense hidden Layer
    hidden = Dense(units=20, activation='relu')(x)

    # Output Layer: 2 neurons (1 for rectangles, 1 for circles), Linear activation function for regression
    outputs = Dense(units=2, activation='linear')(hidden)

    simple_model = Model(inputs=inputs, outputs=outputs)
    simple_model.compile(loss='mse', optimizer='adam', metrics=['mae'])

    return simple_model

CountModel = CounterModel()
CountModel.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 64, 64, 3)	0
conv2d_4 (Conv2D)	(None, 60, 60, 5)	380
max_pooling2d_4 (MaxPooling2D)	(None, 20, 20, 5)	0
conv2d_5 (Conv2D)	(None, 18, 18, 3)	138
max_pooling2d_5 (MaxPooling2D)	(None, 6, 6, 3)	0
flatten_2 (Flatten)	(None, 108)	0
dense_4 (Dense)	(None, 20)	2,180
dense_5 (Dense)	(None, 2)	42

Total params: 2,740 (10.70 KB)

Trainable params: 2,740 (10.70 KB)

Non-trainable params: 0 (0.00 B)

Let's try to mix an image with circles and rectangles and see how the network would evaluate it. First, we will train the network.

```
In [15]: x_train, x_test, y_train, y_test = train_test_split(images, targets, test_size=0.2, random_state=42)
```

```
In [16]: train_output_count = CountModel.fit(x_train, y_train, epochs=100, validation_split=0.2, verbose=0)
```

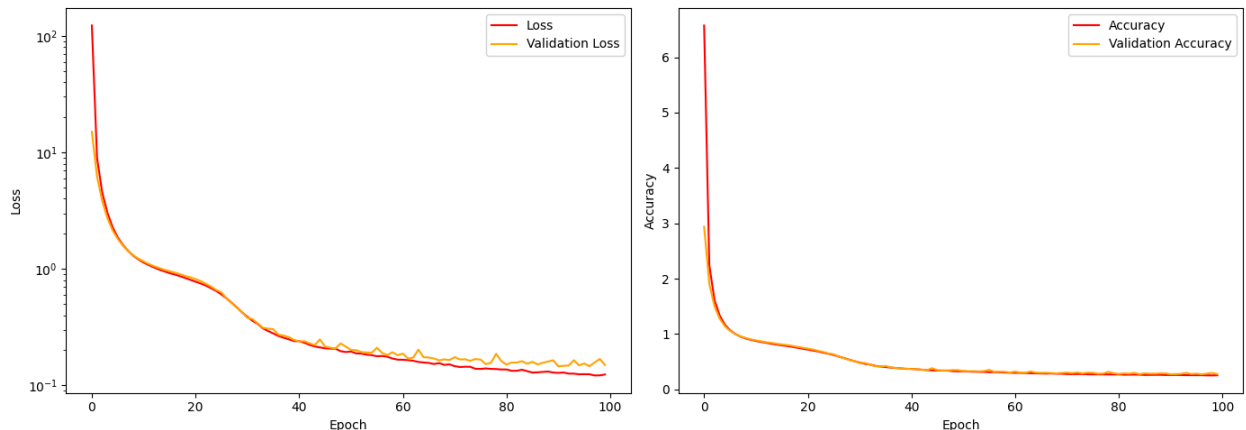
```
In [17]: print(train_output_count.history.keys())
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(train_output_count.history["loss"], color="red", label="Loss")
ax1.plot(train_output_count.history["val_loss"], color="orange", label="Validation Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Loss")
ax1.set_yscale('log')
ax1.legend()

ax2.plot(train_output_count.history["mae"], color="red", label="Accuracy")
ax2.plot(train_output_count.history["val_mae"], color="orange", label="Validation Accuracy")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Accuracy")
ax2.legend()

plt.tight_layout()
plt.show()
```

dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])



Now let's test the performance of our algorithm.

```
In [18]: prediction_counter = CountModel.predict(x_test)
```

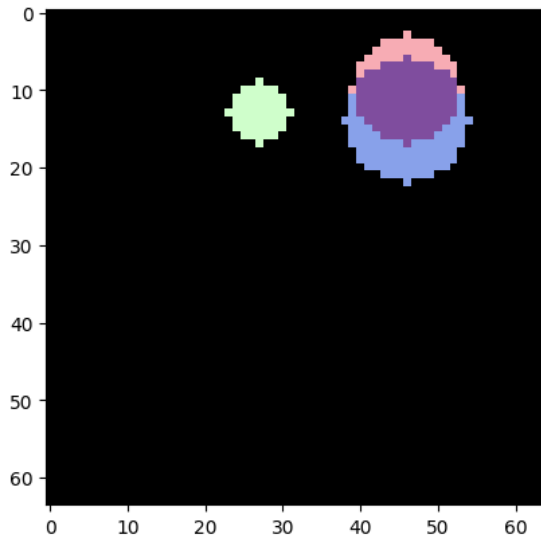
63/63 ————— 1s 6ms/step

```
In [19]: plt.imshow(x_test[0])
plt.show()
print('The label for this case is:', y_test[0])
print(f"According to the prediction, there are {prediction_counter[0][0]} rectangles and {prediction_counter[0][1]} circles")

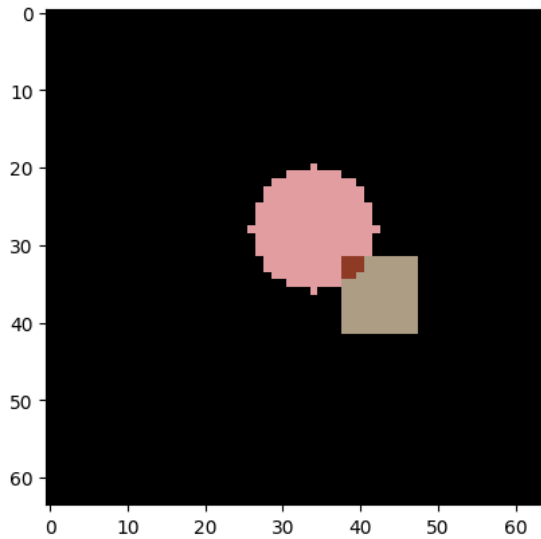
plt.imshow(x_test[12])
plt.show()
print('The label for this case is:', y_test[12])
print(f"According to the prediction, there are {prediction_counter[12][0]} rectangles and {prediction_counter[12][1]} circles")

plt.imshow(x_test[25])
plt.show()
```

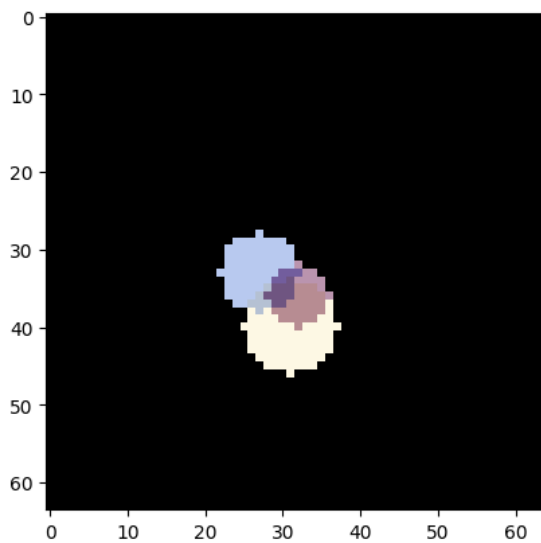
```
print('The label for this case is:', y_test[25])
print(f"According to the prediction, there are {prediction_counter[25][0]} rectangles and {prediction_counter[25][1]} circles")
```



The label for this case is: [0 3]
According to the prediction, there are 0.22300955653190613 rectangles and 2.1338467597961426 circles



The label for this case is: [1 1]
According to the prediction, there are 0.7116293907165527 rectangles and 1.212353229522705 circles



The label for this case is: [0 3]
According to the prediction, there are 0.5111079216003418 rectangles and 1.6640926599502563 circles

Note that in this case it is not easy to choose a metric that describes how well our CNN is working.

CNN and bounding boxes

Even if here the problem is "trivial", it is interesting to see how this network work with a problem where the model has to classify the shape and to provide the bounding box coordinates (detection problem). We will create the data, concatenate the classification (targets) and the detection (boundingBoxes) and then change the previous networks according to the new problem.

As always, let's start by generating some data.

```
In [20]: nsamples=10000

def genwithBB():
    targets=np.array([np.random.rand()>0.5 for x in range(nsamples) ])
    imagesWithBB=[genCircle() if targets[x] else genRectangle() for x in range(nsamples)]
    images=np.array([imagesWithBB[x][0] for x in range(nsamples)])
    boundingBoxes=np.array([imagesWithBB[x][1:] for x in range(nsamples)])

    labels=[targets,boundingBoxes]
    data=images
    return data,labels
data, labels = genwithBB()
```

```
In [21]: import matplotlib.pyplot as plt
import matplotlib.patches as patches

print(labels[0][23])
fig, ax = plt.subplots(1)

ax.imshow(data[23])

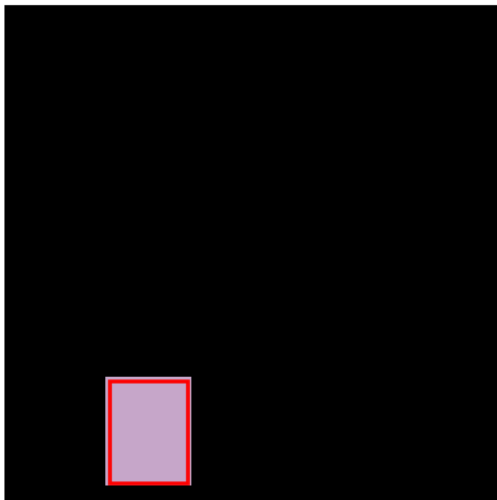
# Extract bounding box coordinates from 'Labels[1]'
bbox_coords = labels[1][23]

# Create a rectangle patch
bbox = patches.Rectangle((bbox_coords[0], bbox_coords[1]),
                        bbox_coords[2] - bbox_coords[0],
                        bbox_coords[3] - bbox_coords[1],
                        linewidth=2, edgecolor='red', facecolor='none')

# Add the patch to the Axes
ax.add_patch(bbox)

plt.axis('off')
plt.show()
```

False



Now let's build the CNN...

```
In [22]: import keras

keras.backend.clear_session()

def BBModel():
    # Input Layer
    inputs = Input(shape=(64, 64, 3))

    # Shared hidden layers for feature extraction
    x = Conv2D(filters=5, kernel_size=(5, 5), activation='relu')(inputs)
    x = MaxPooling2D(pool_size=(3, 3))(x)
    x = Conv2D(filters=3, kernel_size=(3, 3), activation='relu')(x)
    x = MaxPooling2D(pool_size=(3, 3))(x)
    hidden = Flatten()(x)

    # Output 1: Classification (1 neuron, sigmoid for binary choice)
    output1 = Dense(units=1, activation='sigmoid', name='classification')(hidden)

    # Output 2: Regression (4 neurons for bounding box: x_min, y_min, x_max, y_max)
```

```

output2 = Dense(units=4, activation='linear', name='regression')(hidden)

# Group outputs
outputs = [output1, output2]

# Assign specific loss functions for each output branch
losses = {
    'classification': 'binary_crossentropy',
    'regression': 'mse'
}

# Assign specific metrics to monitor during training
metrics = {
    'classification': 'accuracy',
    'regression': 'mse'
}

bb_model = Model(inputs=inputs, outputs=outputs)
bb_model.compile(optimizer='adam', loss=losses, metrics=metrics)

return bb_model

BoundingBoxModel = BBModel()
BoundingBoxModel.summary()

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 64, 64, 3)	0	-
conv2d (Conv2D)	(None, 60, 60, 5)	380	input_layer[0][0]
max_pooling2d (MaxPooling2D)	(None, 20, 20, 5)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 18, 18, 3)	138	max_pooling2d[0]...
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 3)	0	conv2d_1[0][0]
flatten (Flatten)	(None, 108)	0	max_pooling2d_1[...
classification (Dense)	(None, 1)	109	flatten[0][0]
regression (Dense)	(None, 4)	436	flatten[0][0]

Total params: 1,063 (4.15 KB)

Trainable params: 1,063 (4.15 KB)

Non-trainable params: 0 (0.00 B)

... and then complete the fit.

```
In [23]: train_output_bb = BoundingBoxModel.fit(data, labels, validation_split=0.2, epochs=50, verbose=0)
```

Take a look at the exact metric names generated by Keras in this case, then print loss and metric curves.

```
In [24]: print(train_output_bb.history.keys())

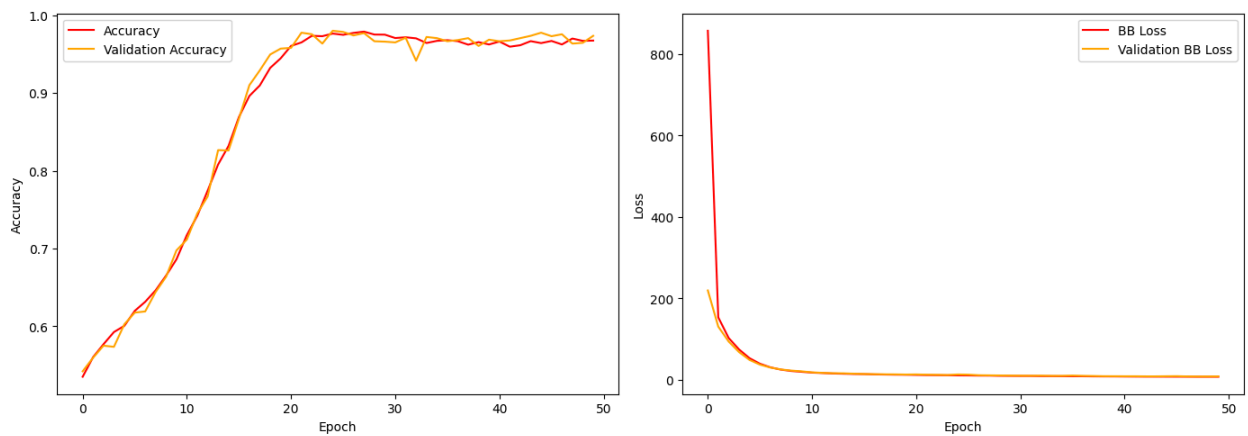
dict_keys(['classification_accuracy', 'classification_loss', 'loss', 'regression_loss', 'regression_mse', 'val_classification_acc
uracy', 'val_classification_loss', 'val_loss', 'val_regression_loss', 'val_regression_mse'])
```

```
In [25]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(train_output_bb.history['classification_accuracy'], color='red', label='Accuracy')
ax1.plot(train_output_bb.history['val_classification_accuracy'], color='orange', label='Validation Accuracy')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy')
ax1.legend()

ax2.plot(train_output_bb.history['regression_loss'], color='red', label='BB Loss')
ax2.plot(train_output_bb.history['val_regression_loss'], color='orange', label='Validation BB Loss')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Loss')
#ax2.set_yscale('log')
ax2.legend()

plt.tight_layout()
plt.show()
```



Let's test our model, but this time instead of using train test split, we sample a new set using the function that originated our data. You can choose the number of samples you like.

```
In [26]: n_testsamples=32
targets=np.array([np.random.rand()>0.5 for x in range(n_testsamples)])
imagesWithBB=[genCircle() if targets[x] else genRectangle() for x in range(n_testsamples)]
images=np.array([imagesWithBB[x][0] for x in range(n_testsamples)])
boundingBoxes=np.array([imagesWithBB[x][1:] for x in range(n_testsamples)])

labels_test=[targets,boundingBoxes]
data_test=images
```

```
In [27]: prediction_bb = BoundingBoxModel.predict(data_test)
```

1/1 ————— 0s 282ms/step

```
In [28]: # Notice that labels_test[0] are the Labels for the classification part
classification_performance = classification_report(labels_test[0], np.round(prediction_bb[0]))
print(classification_performance)
```

	precision	recall	f1-score	support
False	1.00	1.00	1.00	18
True	1.00	1.00	1.00	14
accuracy			1.00	32
macro avg	1.00	1.00	1.00	32
weighted avg	1.00	1.00	1.00	32

Take a look at a single example. Our model seems to work pretty well.

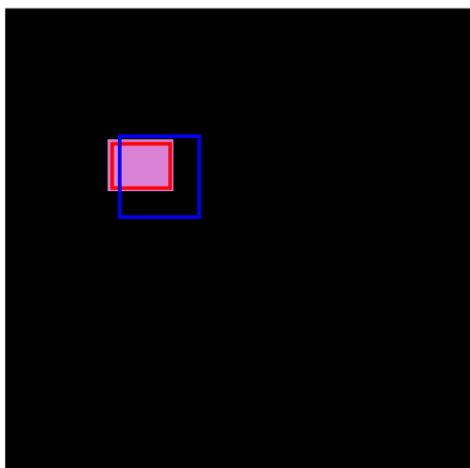
```
In [29]: num = 6
print(labels_test[0][num])
ig, ax = plt.subplots(1)
ax.imshow(data_test[num])

# Create a rectangle patch
bbox = patches.Rectangle((labels_test[1][num][0], labels_test[1][num][1], labels_test[1][num][2] - labels_test[1][num][0], labels_test[1][num][3] - labels_test[1][num][1]))
bbox_pred = patches.Rectangle((int(prediction_bb[1][num][0]), int(prediction_bb[1][num][1]), int(prediction_bb[1][num][2]) - int(prediction_bb[1][num][0]), int(prediction_bb[1][num][3]) - int(prediction_bb[1][num][1])))

# Add the patch to the Axes
ax.add_patch(bbox)
ax.add_patch(bbox_pred)

# Save the result
plt.axis('off')
plt.show()
```

False



Bonus

The following code can be used to visualize what features the first convolutional layer is looking at. Try to understand how it works.

```
In [30]: import matplotlib.pyplot as plt

# Retrieve weights from the first convolutional layer (index 1)
filters, biases = SimModel.layers[1].get_weights()

# Normalize filter values to 0-1 so we can visualize them properly
f_min, f_max = filters.min(), filters.max()
filters = (filters - f_min) / (f_max - f_min)

# Plot the first few filters
n_filters = 5
ix = 1

# Loop over the filters
for i in range(n_filters):
    # Get the i-th filter (shape: height, width, channels)
    f = filters[:, :, :, i]

    # Plot each channel (Red, Green, Blue) separately
    for j in range(3):
        # Specify the subplot for the channel and turn off axis
        ax = plt.subplot(n_filters, 4, ix)
        ax.set_xticks([])
        ax.set_yticks([])
        # Plot the filter channel using the corresponding colormap
        plt.imshow(f[:, :, j], cmap=['Reds', 'Greens', 'Blues'][j])
        ix += 1

    # Plot the combined RGB filter (OUTSIDE the channel loop)
    ax = plt.subplot(n_filters, 4, ix)
    ax.set_xticks([])
    ax.set_yticks([])
    plt.imshow(f)
    ix += 1

# Show the complete figure
plt.tight_layout()
plt.show()
```

